



SILIGURI INSTITUTE OF TECHNOLOGY

IoT-BASED SMART HOME AUTOMATION AND WATER MANAGEMENT SYSTEM

Industrial Internship Project Report

Submitted by

PRATHAM BIKRAM SAHI

Department of Electrical Engineering

Siliguri Institute of Technology



Training Organisation: Ardent Computech Pvt. Ltd.

Project Supervisor: SK Suman

Project Period: 7–8 March 2024

**Ardent® Computech Pvt. Ltd.**Module – 132, Ground Floor, SDF Building,
Sector V, Salt Lake, Kolkata – 700091.

☎ +91 33 40073507

✉ training@ardentcollaborations.com

Acknowledgement

The successful completion of this project was made possible by the guidance, encouragement and cooperation of many individuals. We express our sincere gratitude to everyone who supported us during the development of the IoT-Based Smart Home Automation and Water Management System.

We extend our heartfelt thanks to our project supervisor, **Mr. SK Suman**, for his valuable guidance, constructive suggestions and continuous encouragement throughout the project work.

We are also grateful to the faculty members and technical team of **Ardent Computech Pvt. Ltd.** for providing the opportunity, resources and technical support required to complete this industrial internship project. We also thank the Department of Electrical Engineering, Siliguri Institute of Technology, and everyone who directly or indirectly contributed to the successful completion of this work.

Signature

Date: 12/03/2024

Executive Summary

This project presents an integrated Internet of Things (IoT) solution consisting of two practical modules: a **Smart Home Automation System** for remote appliance control and a **Smart Water Management System** for monitoring water level and automating pump operation. Both modules use the ESP32 microcontroller as the central embedded platform because of its integrated Wi-Fi capability and flexible GPIO interfaces.

The home automation module uses a relay to switch an electrical load through commands received from the Blynk IoT platform. The water management module uses an ultrasonic sensor to estimate tank water level and automatically controls a water pump through a relay according to predefined low- and high-level thresholds. Together, the modules demonstrate how a common IoT architecture can improve convenience, resource management and automation.

Project Objectives

- Develop an ESP32-based IoT platform for practical household automation.
- Enable remote ON/OFF control of an electrical appliance.
- Monitor water level in a storage tank without direct contact with the water.
- Automatically start and stop a water pump according to tank level.
- Demonstrate cloud connectivity, sensor interfacing and relay-based control in one integrated project.

Key Outcomes

- Successful Wi-Fi-enabled appliance switching.
- Automatic water-level-based pump control.
- Real-time water-level measurement suitable for IoT monitoring.
- Practical experience with ESP32, sensors, relays and cloud communication.
- A scalable foundation for future smart-home and resource-management applications.

Table of Contents

Acknowledgement

Executive Summary

1. Introduction

2. Overall System Architecture

Part I — Smart Home Automation System

3. Home Automation Working Principle

4. Hardware and Software

5. Circuit Implementation and Firmware Logic

6. Home Automation Testing and Results

Part II — Smart Water Management System

7. Introduction to Smart Water Management

8. Objectives and Working Principle

9. Hardware Requirements

10. Water-Level Measurement and Pump Control

11. Circuit Connections

12. Smart Water Management Firmware

13. Testing and Expected Results

14. Integrated Project Advantages and Limitations

15. Future Scope

16. Conclusion

17. References

Appendix A: Home Automation Code

Appendix B: Smart Water Management Code

1. Introduction

The Internet of Things connects physical devices, sensors and actuators to digital networks so that real-world processes can be monitored and controlled intelligently. In domestic applications, IoT can improve convenience through appliance automation and can also support better resource management.

This project combines two related applications under a common ESP32-based platform. The first module controls an electrical appliance remotely through Wi-Fi. The second module monitors the water level of a storage tank and automatically controls a pump. The combined project therefore demonstrates both **remote control** and **sensor-based automatic control**.

2. Overall System Architecture

The ESP32 acts as the central embedded controller. Depending on the module, it receives a cloud command or a sensor measurement, processes the information and controls a relay-connected load.



PART I — SMART HOME AUTOMATION SYSTEM

3. Home Automation Working Principle

The user sends an ON/OFF command through the Blynk interface. The ESP32 receives the required control state over Wi-Fi and changes a GPIO output. The relay module then switches the connected electrical load.



4. Hardware and Software

Component / Platform	Purpose
ESP32 development board	Main controller and Wi-Fi interface
5 V relay module	Electrical load switching
Breadboard and jumper wires	Prototype interconnection
Bulb / demonstration load	Output appliance
Arduino IDE	Firmware development
Blynk IoT platform	Remote control and cloud interface

5. Circuit Implementation and Firmware Logic

A selected ESP32 GPIO pin is connected to the relay input. The firmware connects the ESP32 to Wi-Fi, reads the remote control state and updates the relay output. Credentials and authentication tokens must be stored securely and are represented by placeholders in the code included in this report.

- Initialize GPIO and serial communication.
- Connect the ESP32 to Wi-Fi.
- Read the remote command from the IoT service.
- Switch the relay according to the received state.
- Repeat while maintaining network connectivity.

6. Home Automation Testing and Results

The prototype demonstrated successful Wi-Fi connectivity, remote command reception, GPIO control and relay switching. The connected demonstration load could be operated through the IoT interface, validating the basic home automation concept.

PART II — SMART WATER MANAGEMENT SYSTEM

7. Introduction to Smart Water Management

Water storage tanks are commonly filled using electric pumps. When pump operation depends entirely on manual observation, the tank may overflow or the pump may run unnecessarily. A smart water management system can continuously estimate tank level and automatically control the pump, reducing the need for manual intervention.

In this project, an ultrasonic sensor is positioned at the top of the tank to measure the distance between the sensor and the water surface. The ESP32 converts this distance into an estimated water-level percentage and uses predefined thresholds to control a pump through a relay.

8. Objectives and Working Principle

Objectives

- Measure tank water level without direct electrical contact with the water.
- Display or transmit the estimated water-level percentage.
- Automatically start the pump when the water level becomes low.
- Automatically stop the pump before the tank overflows.
- Provide a design that can later be connected to an IoT dashboard for remote monitoring.

Working Principle



The ultrasonic sensor emits a sound pulse and measures the echo return time. The ESP32 calculates the distance to the water surface. If the tank depth is known, the filled-water height can be estimated as the tank depth minus the measured empty distance. The result is converted into a percentage.

9. Hardware Requirements

Component	Function
ESP32 development board	Processes sensor data and controls the system
HC-SR04 ultrasonic sensor	Measures distance to the water surface
5 V relay module	Controls the water pump
Water pump / demonstration load	Represents the filling mechanism
Breadboard and jumper wires	Prototype connections
Suitable power supply	Powers controller, sensor and relay

Practical note: The HC-SR04 Echo output is typically 5 V, while ESP32 GPIO uses 3.3 V logic. A suitable voltage divider or level-shifting arrangement should be used on the Echo connection.

10. Water-Level Measurement and Pump Control

For a tank of known usable depth **D** and measured sensor-to-water distance **d**, the approximate filled height is **D – d**. The water-level percentage can therefore be estimated as:

Water Level (%) = [(D – d) / D] × 100

To prevent rapid relay switching around a single threshold, the control logic uses two thresholds. In the example implementation, the pump starts when the level falls to 25% or below and stops when the level reaches 90% or above. Between these limits, the previous pump state is maintained. This two-threshold method provides simple hysteresis.

11. Circuit Connections

ESP32 / Device	Connection
GPIO 5	HC-SR04 TRIG
GPIO 18	HC-SR04 ECHO through suitable level reduction
GPIO 23	Relay IN
5 V / appropriate supply	Sensor and relay supply according to module requirements
GND	Common low-voltage reference
Relay COM and NO	Pump/load switching circuit

Electrical safety: A mains-powered pump must be installed with correctly rated switching equipment, isolation, protection and enclosure. The low-voltage prototype control circuit should not be treated as a complete mains installation design.

12. Smart Water Management Firmware

The firmware periodically triggers the ultrasonic sensor, calculates the water-level percentage and applies automatic pump-control logic. The code is written so that the tank depth and control thresholds can be adjusted for the actual prototype.

```
// ESP32 Smart Water Management System
#define TRIG_PIN 5
#define ECHO_PIN 18
#define RELAY_PIN 23

const float TANK_DEPTH_CM = 100.0;
const float PUMP_ON_LEVEL = 25.0;
const float PUMP_OFF_LEVEL = 90.0;

bool pumpOn = false;

float readDistanceCM() {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);

    long duration = pulseIn(ECHO_PIN, HIGH, 30000);
    if (duration == 0) return -1;

    return duration * 0.0343 / 2.0;
}

void setPump(bool state) {
    pumpOn = state;
    // Many relay modules are active LOW.
    digitalWrite(RELAY_PIN, state ? LOW : HIGH);
}

void setup() {
    Serial.begin(115200);
    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);
    pinMode(RELAY_PIN, OUTPUT);
    setPump(false);
}

void loop() {
    float distance = readDistanceCM();

    if (distance > 0 && distance <= TANK_DEPTH_CM) {
        float level =
            ((TANK_DEPTH_CM - distance) / TANK_DEPTH_CM) * 100.0;

        level = constrain(level, 0, 100);

        if (level <= PUMP_ON_LEVEL) {
            setPump(true);
        } else if (level >= PUMP_OFF_LEVEL) {
            setPump(false);
        }

        Serial.print("Water Level: ");
        Serial.print(level, 1);
        Serial.print("% | Pump: ");
        Serial.println(pumpOn ? "ON" : "OFF");
    } else {
        Serial.println("Sensor reading invalid.");
    }

    delay(1000);
}
```

13. Testing and Expected Results

The water-management module is designed to provide continuous level estimation and automatic pump control. During prototype testing, the measured distance should decrease as the water level rises and increase as the level falls. The calculated percentage should follow this change. When the level reaches the lower threshold, the relay activates the pump; when the upper threshold is reached, the relay deactivates it.

- Low water level → pump automatically turns ON.
- Intermediate level → previous pump state is maintained.
- High water level → pump automatically turns OFF.
- Invalid sensor reading → no new automatic switching command is issued in the example logic.

Actual accuracy depends on tank geometry, sensor placement, surface movement and calibration. The tank-depth constant and thresholds should therefore be adjusted to match the physical prototype.

14. Integrated Project Advantages and Limitations

Advantages

- Combines remote appliance control and automatic resource management.
- Uses a common ESP32 platform for multiple IoT applications.
- Reduces manual intervention in appliance and pump operation.
- Can be expanded with additional sensors, loads and cloud dashboards.
- Demonstrates both actuator control and real-time sensor processing.

Limitations

- Internet-based features depend on network availability.
- Ultrasonic readings can be affected by mounting and tank conditions.
- Relay switching must be designed safely for the actual load.
- A prototype requires further protection and enclosure for permanent deployment.

15. Future Scope

- Display both appliance status and tank level on one IoT dashboard.
- Add notifications for low water, full tank and abnormal pump operation.
- Add flow sensors to measure actual water consumption.
- Detect leakage using flow and level data.
- Add dry-run protection for the pump.
- Store historical water-level and consumption data.

Appendix A: Sanitized Home Automation Code

```
#include <WiFi.h>
#include <HTTPClient.h>

const char* ssid = "YOUR_WIFI_SSID";
const char* password = "YOUR_WIFI_PASSWORD";
const int relayPin = 23;
String url = "YOUR_BLYNK_API_ENDPOINT";

void setup() {
    Serial.begin(9600);
    pinMode(relayPin, OUTPUT);
    WiFi.begin(ssid, password);
}

void loop() {
    if (WiFi.status() == WL_CONNECTED) {
        HTTPClient http;
        http.begin(url);
        int httpCode = http.GET();

        if (httpCode == 200) {
            String state = http.getString();
            digitalWrite(relayPin, state == "1" ? LOW : HIGH);
        }
        http.end();
    }
    delay(500);
}
```

Appendix B: Smart Water Management Code

```
// ESP32 Smart Water Management System
#define TRIG_PIN 5
#define ECHO_PIN 18
#define RELAY_PIN 23

const float TANK_DEPTH_CM = 100.0;
const float PUMP_ON_LEVEL = 25.0;
const float PUMP_OFF_LEVEL = 90.0;

bool pumpOn = false;

float readDistanceCM() {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);

    long duration = pulseIn(ECHO_PIN, HIGH, 30000);
    if (duration == 0) return -1;

    return duration * 0.0343 / 2.0;
}

void setPump(bool state) {
    pumpOn = state;
    // Many relay modules are active LOW.
    digitalWrite(RELAY_PIN, state ? LOW : HIGH);
}

void setup() {
    Serial.begin(115200);
    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);
    pinMode(RELAY_PIN, OUTPUT);
    setPump(false);
}

void loop() {
    float distance = readDistanceCM();

    if (distance > 0 && distance <= TANK_DEPTH_CM) {
        float level =
            ((TANK_DEPTH_CM - distance) / TANK_DEPTH_CM) * 100.0;

        level = constrain(level, 0, 100);

        if (level <= PUMP_ON_LEVEL) {
            setPump(true);
        } else if (level >= PUMP_OFF_LEVEL) {
            setPump(false);
        }

        Serial.print("Water Level: ");
        Serial.print(level, 1);
        Serial.print("% | Pump: ");
        Serial.println(pumpOn ? "ON" : "OFF");
    } else {
        Serial.println("Sensor reading invalid.");
    }

    delay(1000);
}
```

- Use secure authentication and encrypted communication.
- Integrate solar-powered pumping or energy monitoring.

16. Conclusion

The IoT-Based Smart Home Automation and Water Management System demonstrates two practical applications of embedded control using the ESP32. The home automation module enables remote switching of an electrical appliance through an IoT platform, while the smart water management module measures tank level and automatically controls a pump using threshold-based logic.

By combining cloud communication, sensor interfacing and relay control in one project, the work provides a broader understanding of IoT-based automation. The system can be further developed into a more complete smart-home platform with energy monitoring, water-consumption analysis, alerts and additional safety features.

17. References

Espressif Systems — ESP32 technical documentation and datasheets.

Arduino — Arduino IDE documentation.

Blynk — IoT platform documentation.

HC-SR04 ultrasonic sensor technical documentation.

Relay module and power-supply datasheets used for the prototype.